

Рекурсивный вызов функций Макроопределения

Основы языка C, лекция 3

Рекуррентное определение функций

$$n! = \begin{cases} 1 & n = 0 \\ n \cdot (n - 1)! & n > 0 \end{cases}$$

$$f(n) = \begin{cases} 1 & n = 0 \\ n \cdot f(n - 1) & n > 0 \end{cases}$$

Рекурсивный вызов функций

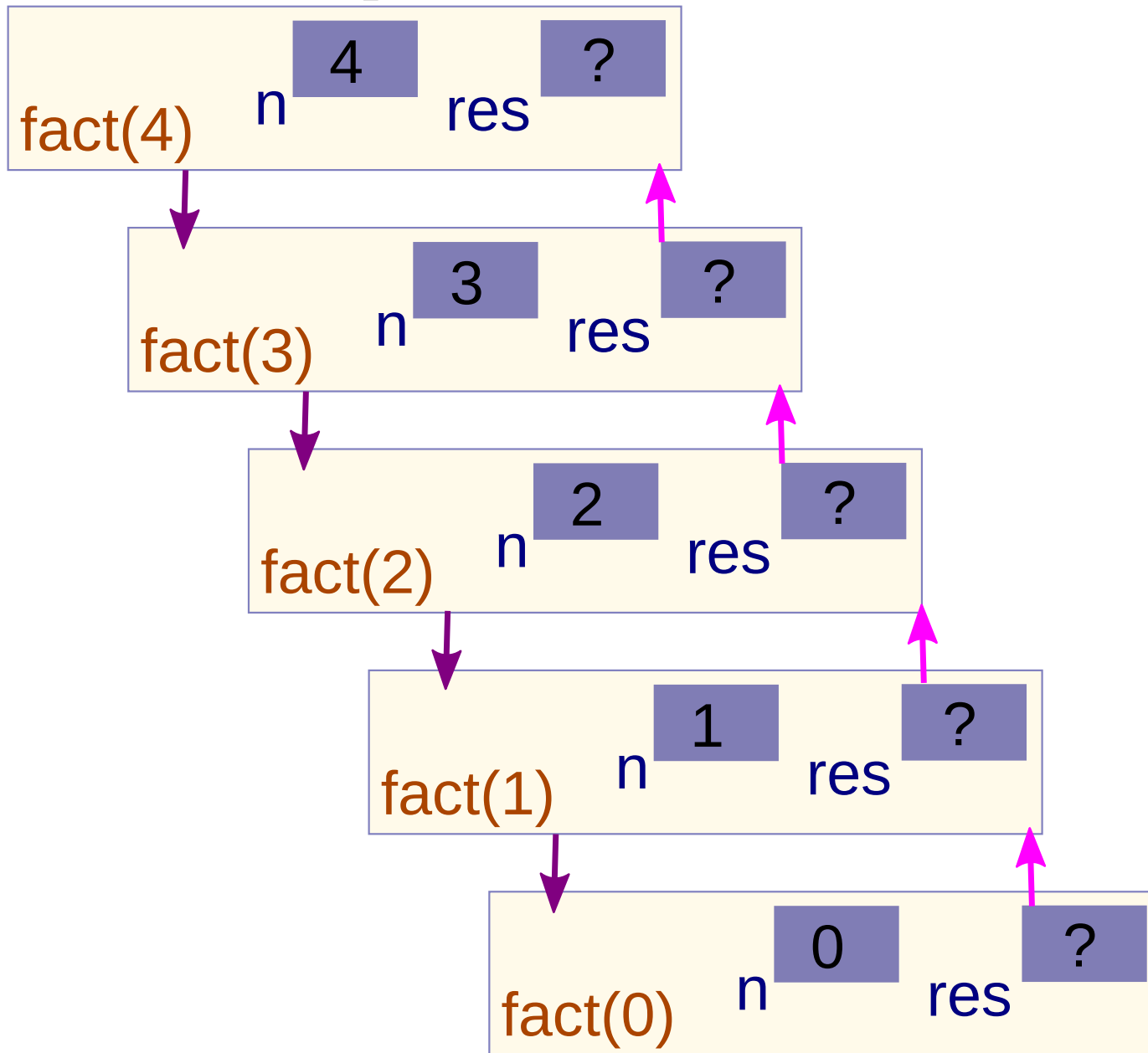
- $$// n! = n * (n-1)!$$

```
int fact (int n) {  
    int res = n * fact (n-1);    // рекурсия  
    return res;  
}  
  
int main ( ) {  
    printf("4! = %d\n", fact(4));  
    return 0;  
}
```

Не работает. Почему?

Segmentation fault (core dumped)

Экземпляры n и res



Отладочная печать

- `#include <stdio.h>`

```
int fact (int n) {  
    printf("call fact(%d)\n", n);  
    int res = n * fact (n-1);  
    printf("%d! = %d\n", n, res);  
    return res;  
}  
  
int main ( ) {  
    printf("4! = %d\n", fact(4));  
    return 0;  
}
```

- call fact(4)
- call fact(3)
- call fact(2)
- call fact(1)
- call fact(0)
- call fact(-1)
- call fact(-2)
- call fact(-3)
- call fact(-4)
- call fact(-5)

...

$0! = 1$

- Если $n=0$, то значение $n!=1$
- **if** (*условие*)
оператор;
- Если условие истинно,
то выполняется оператор

Если ложно, то оператор не выполняется

- условие пишем в круглых скобках
- табуляция — уровень вложенности для оператора
- блочный оператор **{** `x = 2; printf("Hello\n");` **}**

Операторы сравнения

Математика	Язык C	Русский
$>$	$>$	больше
$\geq$	$>=$	больше или равно
$<$	$<$	меньше
$\leq$	$<=$	меньше или равно
$=$	$==$	равно
$\neq$	$!=$	не равно

Не путайте $x = 5$ и $x == 5$
Чаще присваивают, поэтому 1 равенство

if else - альтернатива

- Проверим число четное или нечетное:

- **if** ($x \% 2 == 0$) {
 printf("четное (even)\n");
} **else** {
 printf("НЕ четное (odd)\n");
}

- **if** (условие) {
 операторДА;
} **else** {
 операторНЕТ;
}

x	x / 2	x % 2
8	4	0
-8	-4	0
7	3	1
-7	-3	-1

F(0) = 1

- ```
int fact (int n) {
 printf("call fact(%d)\n", n);
 if (n == 0) {
 printf("0! = 1\n");
 return 1;
 }
 int res = n * fact (n-1);
 printf("%d! = %d\n", n, res);
 return res;
}

int main () {
 printf("4! = %d\n", fact(4));
 ...
```

- call fact(4)
- call fact(3)
- call fact(2)
- call fact(1)
- call fact(0)
- 0! = 1
- 1! = 1
- 2! = 2
- 3! = 6
- 4! = 24
- main: 4! = 24

# Печать с отступами

- `int fact (int n, int depth)`  
    `printf("%-*c call fact(%d)\n", depth, '>', n);`  
    `if (n == 0) {`  
        `printf("%-*c 0! = 1\n", depth, '>');`  
        `return 1;`  
    `}`  
    `int res = n * fact (n-1, depth+1);`  
    `printf("%-*c %d! = %d\n", depth, '>', n, res);`  
    `return res;`  
}

- `fact (4, 1)`

# Печать с отступами

```
> call fact(4)
> call fact(3)
> call fact(2)
> call fact(1)
> call fact(0)
> 0! = 1
> 1! = 1
> 2! = 2
> 3! = 6
> 4! = 24
main: 4! = 24
```

```
• int fact (int n) {
 if (n == 0)
 return 1;
 return n * fact (n-1);
}
int main() {
 printf("main: 4! = %d\n",
 fact(4));
 return 0;
}
```

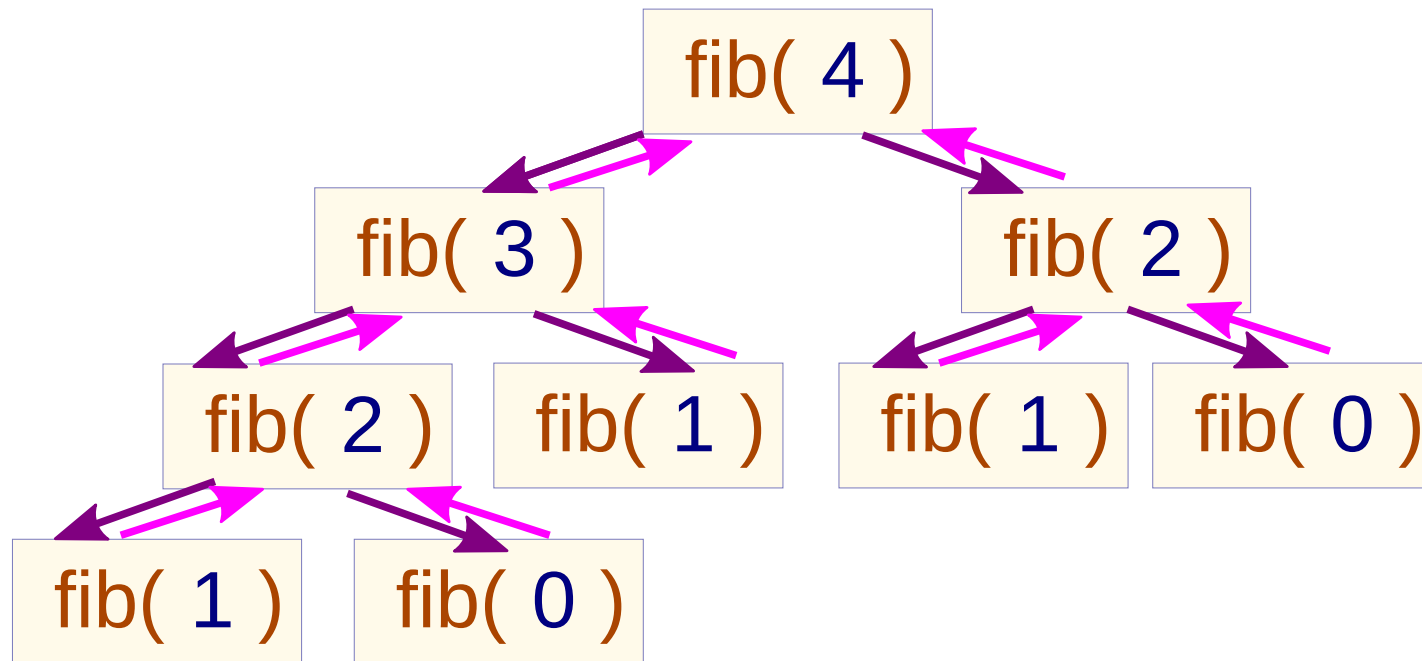
# Числа Фибоначчи

$$\begin{cases} fib(0) = fib(1) = 1 \\ fib(n) = fib(n-1) + fib(n-2) & n > 1 \end{cases}$$

- 0 1 2 3 4 5 6 7  
1 1 2 3 5 8 13 21 ...
- ```
int fib (int n) {  
    if (n == 0) return 1;    // if (n==0 || n== 1)  
    if (n == 1) return 1;    //      return 1;  
    return fib(n-1) + fib(n-2);  
}  
  
int main () {  
    printf("fib(5) = %d\n", fib(5));  
}
```

ЭФФЕКТИВНОСТЬ

- Сколько раз будет вызвано `fib(2)` при подсчете `fib(20)` ?
fib(n-1) + **fib**(n-2)



НОД и НОК

- НОД — наибольший общий делитель для 2 чисел

$$\text{НОД}(54, 24) = 6$$

$$54 = 9 * 6$$

$$24 = 4 * 6$$

$$\text{НОД}(9, 20) = 1 \text{ — взаимно простые числа}$$

- НОК — наименьшее общее кратное

$$\text{НОК}(54, 24) = 9 * 6 * 4 = 216$$

$$\text{НОК}(16, 20) = 80$$

- $a * b = \text{НОД}(a, b) * \text{НОК}(a, b)$

Алгоритм Евклида НОД

- НОД (123, 21)

$$123 \% 21 = 18$$

$$21 \% 18 = 3$$

$$18 \% 3 = 0$$

- НОД (21, 125)

$$21 \% 125 = 21$$

$$125 \% 21 = 20$$

$$21 \% 20 = 1$$

$$20 \% 1 = 0$$

- $\text{НОД}(a,b) \cdot \text{НОК}(a,b) = a \cdot b$

Возведение в степень a^n

- Алгоритм "в лоб" $a^n = a \cdot a^{n-1}$
- $a^{12} \rightarrow a^{11} \rightarrow a^{10} \rightarrow a^9 \rightarrow a^8 \rightarrow a^7 \rightarrow a^6 \rightarrow a^5 \rightarrow a^4 \rightarrow a^3 \rightarrow a^2 \rightarrow a^1$
- Сложность алгоритма $n-1$
- Умный алгоритм $a^{2n} = a^n \cdot a^n$
- $a^8 \rightarrow a^4 \rightarrow a^2 \rightarrow a^1$
- Сложность: число знаков в двоичной записи
- Доработаем для нечетных степеней $a^{2n+1} = a \cdot a^{2n}$
- $a^{12} \rightarrow a^6 \rightarrow a^3 \rightarrow a^2 \rightarrow a^1$
- Число знаков + число единиц в двоичной записи
 $12 = 1100$

Делители

- Напечатать делители числа в виде
 $20 = 2^2 \cdot 5$
- `factor(a, last, n)` // число, делитель, его степень

- | a | last | n | |
|----|------|---|---------------------------------|
| 20 | 2 | 0 | |
| 10 | 2 | 1 | |
| 5 | 2 | 2 | печатать 2^2 |
| 5 | 3 | 0 | |
| 5 | 4 | 0 | |
| 5 | 5 | 0 | |
| 1 | 5 | 1 | печатаем 5 |
| 0 | | | можно обойтись без этого вызова |

`if (a / last == 0)`

Русский крестьянский способ умножения

- только $\times 2$, $/2$, $+$ и $-$

$a \rightarrow a \times 2$

$b \rightarrow b/2$

```
int rusMult(int a, int b);
```

- | a | b | b%2 |
|---|---|-----|
|---|---|-----|

100	37	1
-----	----	---

200	18	0
-----	----	---

400	9	1
-----	---	---

800	4	0
-----	---	---

1600	2	0
------	---	---

3200	1	1
------	---	---

100 37

200 18

400 9

800 4

1600 2

3200 1

3200

Русский крестьянский способ умножения

- только $\times 2$, $/2$, $+$ и $-$
 $a \rightarrow a \times 2$
 $b \rightarrow b/2$

3200 1600 0 3200
3200 800 0 3200
3200 400 1 3600
3600 200 0 3600
3600 100 1 3700

- | a | b | b%2 | res |
|------|----|-----|-------------------------------|
| 100 | 37 | 1 | $3600 + 1 \times 100 = 3700$ |
| 200 | 18 | 0 | $3600 + 0 \times 200 = 3600$ |
| 400 | 9 | 1 | $3200 + 1 \times 400 = 3600$ |
| 800 | 4 | 0 | $3200 + 0 \times 800 = 3200$ |
| 1600 | 2 | 0 | $3200 + 0 \times 1600 = 3200$ |
| 3200 | 1 | 1 | 3200 |

Русский крестьянский способ деления

- `int rusDel(int a, int b);` `// x = rusDel(77, 7);`

- a а четное? что делаем mult rem

- 77 нет —b 1 1

- 70 да /2 2 0

- 35 нет —b 1 1

- 28 да /2 2 0

- 14 да /2 2 0

- 7 нет —b 1 1

- 0 - - - -

70 1 1

35 2 0

28 1 1

14 2 0

7 2 0

Русский крестьянский способ деления

- `int rusDel(int a, int b);` `// x = rusDel(77, 7);`

- `a` четное? делаем `mult rem part`

- 77 нет —b 1 1 $10*1+1=11$

- 70 да /2 2 0 $5*2+0=10$

- 35 нет —b 1 1 $4*1+1=5$

- 28 да /2 2 0 $2*2+0=4$

- 14 да /2 2 0 $1*2+0=2$

- 7 нет —b 1 1 $0*1+1=1$

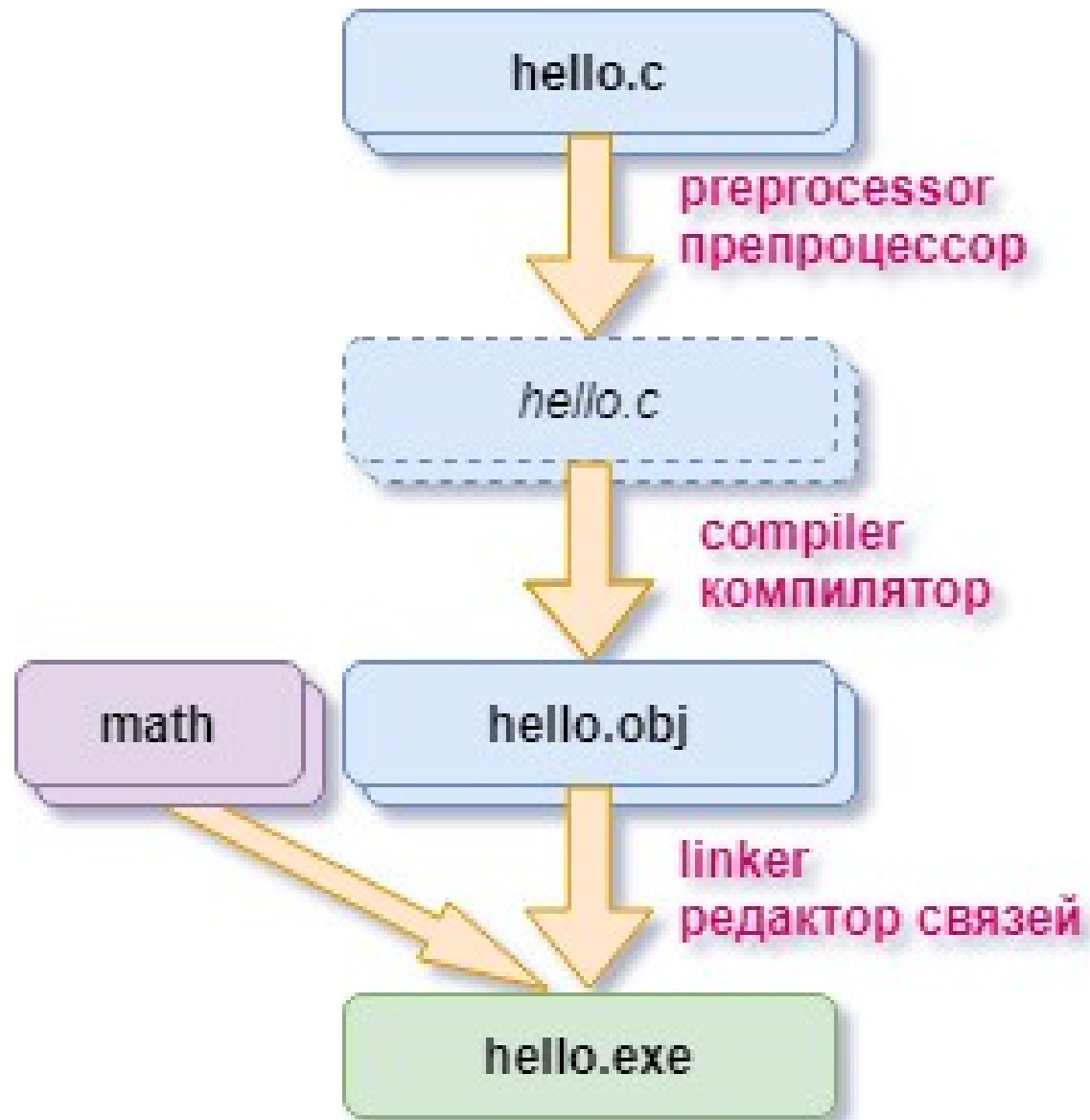
вернули 1

- 0 - - - -

1
 $1 * 2 + 0 = 2$
 $2 * 2 + 0 = 4$
 $4 * 1 + 1 = 5$
 $5 * 2 + 0 = 10$
 $10 * 1 + 1 = 11$
11

Способ деления с остатком

Препроцессор (replace)



include

- Работает до компилятора, # команды
- текстовая замена и условная компиляция
- **#include** <stdio.h> // стандартные библиотеки
Заменить эту строку содержимым файла stdio.h
 - системных директориях
 - указанных явно в командной строке -I
- **#include** "my.h" // свой проект и библиотеки
 - сначала в текущей директории запуска gcc
 - указанных явно в командной строке -I
 - системных директориях

#define - макроопределение

- **#define** идентификатор текстовая_замена
идентификатор — правила C [_a-zA-Z0-9]
текстовая замена — все до конца строки (поможет \)

```
#define A 3
```

```
int x;
```

```
x = A + x;    // x = 3 + x;
```

- Не ставьте в конце ; получите ошибку
#define A 3 ;

```
x = A + x;    // x = 3 ; + x;
```

Макросы с аргументами

- `sin`, `sinf`, `sinl` — синусы для разных типов
- `#define cube(x) x * x * x`
`y = cube(2);`
Подстановка кода, не имеет значение тип
- Время компиляции и время выполнения
- отладка (`debug`)
- Отличие от функции:
 - нет типов
 - текстовая замена VS ВЫЗОВ

Больше скобок!

- `#define sum(a, b) a + b` // будут проблемы!
`y = sum (2, 3) * 4;`
`y = 2 + 3 * 4`
`#define sub(a, b) (a + b)` // так лучше
- `#define square(x) (x * x)`
`int a = 2, b;`
`b = square (a + 3);`
после замены получим:
`b = ( a + 3 * a + 3 );`
- `#define square(x) ( (x) * (x) )`

X++ N ++X

- `int x = 2;`
`printf("%d\n", x++);` // 2
`printf("%d\n", x);` // 3
`printf("%d\n", ++x);` // 4
`printf("%d\n", x);` // 4
- `int y, x = 2;`
`y = ++x + ++x;`
Sequence point
Undefined behavior
- https://en.cppreference.com/w/cpp/language/eval_order
-

Как сломать макрос?

- `#define square(x) ( (x) * (x) )`
- `int x = 2, y;`
`y = square(x++);`
`y = ( (x++) * (x++) );`
- Не используйте вызовы функций в аргументах макроса
`int c = getchar( );`
`isdigit(c = getchar() )`

`#isdigit(x) ('0' <= (x) && (x) <= '9')`

Условная компиляция

- `#define DEBUG`
`#define _WIN64`
- `#ifdef __linux__` // #if defined (__linux__)
много кода, который будет работать в Linux
`#elif defined (_WIN64)`
много кода для 64-битной Windows
`#else`
`printf ("Какая у вас операционная система?\n");`
`#endif`

Если не нужно посылать main

- #define AAA // безумный идентификатор

#ifdef AAA

```
#include <stdio.h>
float a3 (float x);
int main ( ) {
    float z;
    scanf("%f", &z);
    return 0;
}
```

Закомментируем define
// #define AAA
перед посылкой на сервер

или (еще лучше):
gcc **-D**AAA ...
#define AAA писать не нужно

#endif

```
float a3 (float x) {
    return x*x*x;
}
```

Встроенные макросы

- `__LINE__`
- `__FUNCTION__`
- `__FILE__`
- ```
void foo () {
 printf("%d: %s\n", __LINE__, __FUNCTION__);
}
```
- `"бревно" "бревно"`